Driver Drowsiness Detection System Documentation

EL HANNACH Walid

Supervised by:
M.JERROUNDI Taha / Mme.ALAOUI Noura

Capgemini Engineering
MBSE Department
POSTURE PRO Team

August 30, 2024

Abstract

This report presents the development, integration, and testing of an advanced driver assistance system aimed at enhancing road safety through real-time monitoring and intervention. The system combines facial and posture analysis for drowsiness detection with a robust emergency stop mechanism within a simulated environment. Utilizing advanced technologies such as PERCLOS (Percentage of Eye Closure) calculation, posture detection, and the pretrained YOLO model for object recognition, the system offers comprehensive monitoring and timely alerts to prevent fatigue-related accidents. The CARLA simulator's high-fidelity capabilities were employed to evaluate the system's performance under various driving conditions, particularly focusing on emergency scenarios triggered by critical driver fatigue. The report discusses the implementation details, challenges encountered, including multi-threading issues, and the effectiveness of using simulation tools to develop and refine autonomous driving technologies.

"Autonomous vehicles have the potential to significantly improve road safety, reduce traffic congestion, and create new mobility options for people who are unable to drive themselves," says Raj Rajkumar, director of the Carnegie Mellon University Robotics Institute.

Contents

1 Algorithm Specification **5**
- 1.1 PERCLOS Calculation . . . 5
  - 1.1.1 Overview . . . 5
  - 1.1.2 Eye Aspect Ratio (EAR) Calculation . . . 5
  - 1.1.3 Implementation . . . 5
- 1.2 Posture Detection . . . 6
  - 1.2.1 Overview . . . 6
  - 1.2.2 Methodology . . . 6
- 1.3 Pretrained YOLO Model . . . 6
  - 1.3.1 Overview . . . 6
  - 1.3.2 Dataset Preparation . . . 6
  - 1.3.3 Model Training . . . 6

2 Algorithms Development **7**
- 2.1 Yolo Model Training . . . 7
  - 2.1.1 Training Environment . . . 7
  - 2.1.2 Training Process . . . 7
  - 2.1.3 Training Results . . . 7
  - 2.1.4 Model Validation . . . 8
  - 2.1.5 Limitations and Future Improvements . . . 8
- 2.2 Detections Levels . . . 9
- 2.3 Sub-Models Coding Logic . . . 9
  - 2.3.1 Perclos Calculation Coding . . . 9
  - 2.3.2 Posture Detection Coding . . . 10
- 2.4 Integration of the Two Sub-Models . . . 11
  - 2.4.1 Application Overview . . . 11
  - 2.4.2 User Interface and Video Processing . . . 11
  - 2.4.3 Drowsiness Level Assessment . . . 12
  - 2.4.4 Real-Time Operation and Control . . . 12
  - 2.4.5 Summary of Integration . . . 12
- 2.5 Global Integration with Yolo Pretrained Model . . . 12
  - 2.5.1 Functionality Overview . . . 12
  - 2.5.2 Drowsiness Detection Logic . . . 12
  - 2.5.3 Output and User Interaction . . . 13

3 Test and Simulation of Code Logic **14**
- 3.1 Result of the Execution of the Two Sub-Models on a Local Machine . 14
  - 3.1.1 Drowsiness Level Indicator . . . 14
  - 3.1.2 Control Buttons . . . 15
  - 3.1.3 How It Works . . . 15

3.2 Result of Testing Global Program on Google Colab 16
3.2.1 Key Features 16
3.2.2 Functionality 17

4 Widescreen Development **18**
4.1 Widescreen Functionalities 18
4.1.1 User Interface Layout 18
4.1.2 Drowsiness Alerts 18
4.2 Used Tools 19
4.3 Integration of Pop-Ups 19
4.3.1 Dialog Management 19
4.3.2 Alert Levels 19
4.3.3 Blinking and Timers 20
4.3.4 User Interaction and Safety Measures 20
4.4 Test and Simulation 20
4.4.1 Alerts and Dialogs 21

5 Integration on CARLA Simulator **22**
5.1 Tools Explanation 22
5.2 Integration with our main system 23
5.2.1 Connecting to the Simulation 23
5.2.2 Setting Up the Simulation 23
5.2.3 Spawning a New Car 23
5.2.4 Automatic Driving 23
5.2.5 User Interaction 23
5.2.6 Stopping the Car 24
5.2.7 Ending the Simulation 24
5.2.8 Specifications 24
5.3 multi-threading issue 24
5.3.1 Error Overview 24
5.3.2 Potential Problems 25
5.3.3 Recommendations 25

Introduction

The increasing complexity of modern driving environments has amplified the need for advanced driver assistance systems (ADAS) that can detect and respond to potential safety risks in real time. One of the most pressing concerns in this domain is driver drowsiness, a major contributor to traffic accidents globally. This report details the development of a driver monitoring system designed to assess drowsiness through real-time video analysis of facial features and posture, integrated with a responsive emergency stop system. The system's core components—PERCLOS for eye closure measurement, posture detection for head angle analysis, and YOLO for object recognition—are integrated into a user-friendly PyQt5 application. To validate the system's efficacy, the CARLA simulator was utilized, providing a controlled environment to test emergency responses to critical fatigue levels. The integration process, system functionality, and challenges encountered during the development are thoroughly explored, demonstrating the potential of combining real-time monitoring with simulation tools to enhance driver safety.

Chapter 1

Algorithm Specification

1.1 PERCLOS Calculation

The PERCLOS (Percentage of Eye Closure) metric is an essential component of driver drowsiness detection systems. This metric quantifies the percentage of time a driver's eyes are closed over a given period, which is a reliable indicator of drowsiness. The PERCLOS calculation module is developed as follows:

1.1.1 Overview

The module utilizes the `dlib` library for facial landmark detection and `OpenCV` for image processing.

1.1.2 Eye Aspect Ratio (EAR) Calculation

The Eye Aspect Ratio (EAR) is calculated using six key points that represent the landmarks of each eye. The EAR is a critical measure used to determine whether an eye is open or closed. The formula used is:

$$EAR = \frac{\text{dist}(A, B)}{2 \times \text{dist}(C, D)}$$

Where `A`, `B`, `C`, and `D` represent the distances between specific eye landmarks.

1.1.3 Implementation

A Python class `PERCLOSCalculator` is implemented to handle the PERCLOS calculations. The main steps include:

- **Initialization:** The class is initialized with parameters such as the threshold for EAR, the alert threshold for PERCLOS, the time period for analysis, and the frames per second (FPS) of the video feed.
- **EAR Calculation:** For each detected face in a video frame, the EAR for both eyes is computed.
- **PERCLOS Computation:** The percentage of frames where the eyes are closed (i.e., EAR below a predefined threshold) is calculated over the specified time period.

1.2 Posture Detection

The Posture Detection module aims to assess the driver's posture to detect signs of drowsiness, which is a critical part of the overall system. It was developed using `OpenPose` for posture detection.

1.2.1 Overview

The system analyzes the driver's posture by detecting the relative positions of key body landmarks.

1.2.2 Methodology

The module processes video frames to detect human postures. Key steps include:

- **Landmark Detection:** Facial landmarks are detected using the `dlib` library.
- **Posture Analysis:** The detected landmarks are analyzed to determine if the posture indicates signs of drowsiness, such as slumping or head tilt.

1.3 Pretrained YOLO Model

The YOLO (You Only Look Once) model is employed to detect objects in real-time, which includes recognizing driver behaviors indicative of drowsiness.

1.3.1 Overview

The YOLO model was trained using a custom dataset created with the Roboflow API, and then fine-tuned for this specific task. The training process was executed on Google Colab using the ultralytics library.

1.3.2 Dataset Preparation

The dataset was prepared by downloading a project version from Roboflow, which included training, validation, and test sets. The paths to these sets were then correctly configured in a YAML file before starting the model training.

1.3.3 Model Training

A pretrained YOLOv8 model was fine-tuned on the custom dataset for 50 epochs with an image size of 640x640 pixels. The training process was done using the `ultralytics` library, which simplified the integration with the rest of the system.

Chapter 2

Algorithms Development

2.1 Yolo Model Training

2.1.1 Training Environment

The YOLOv8 model was trained using Google Colab's GPU resources. This choice was made due to the lack of a GPU on our local machine, which made it impractical to perform the training locally. Google Colab provides an accessible environment for training deep learning models, especially for tasks requiring significant computational power, such as YOLO.

2.1.2 Training Process

The training was conducted on the custom dataset using the YOLOv8 model. The training code is as follows:

```
from ultralytics import YOLO

# Load a YOLOv8 model
model = YOLO("yolov8n.pt")

# Train the model on the dataset
model.train(data="/content/capg,dda-3/data.yaml", epochs=50, imgsz=640)
```

The model was trained for 50 epochs with an image size of 640x640 pixels. The training process was efficient, with the YOLOv8 model showing continuous improvement across the epochs.

2.1.3 Training Results

The results from the training phase are summarized below. The table shows the performance metrics for the 50th epochs, highlighting key values such as box loss, class loss, and the overall mAP (mean Average Precision).

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
50/50	2.27G	0.4479	0.2156	0.8833	15	640: 100%	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
	all	1841	1841	0.981	0.985	0.992	0.912

Figure 2.1: The performance metrics for the 50th epochs.

After 50 epochs, the model achieved a mAP50 of 0.992 and a mAP50-95 of 0.912, with an overall precision of 0.981 and recall of 0.985. These results indicate that the model is highly accurate in detecting the relevant classes.

2.1.4 Model Validation

Following the completion of the training, the model was validated using the best weights obtained from the training process. The validation results further confirmed the model's performance:

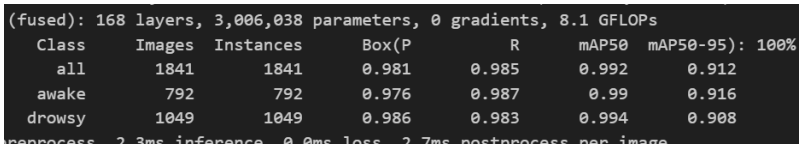


Figure 2.2: Model metrics summary.

The best model weights were saved to a file named `best.pt` for future use in deployment or further fine-tuning.

2.1.5 Limitations and Future Improvements

Although the model achieved impressive accuracy, there is potential for further improvement. For instance, increasing the number of epochs from 50 to 100 could potentially enhance the model's accuracy. However, due to the limitations of our Google Colab environment, specifically the limited access to GPU resources in the free version, we were unable to train for more epochs. A paid Google Colab version or an alternative with more substantial GPU resources would allow for more extensive training and possibly better performance.

2.2 Detections Levels

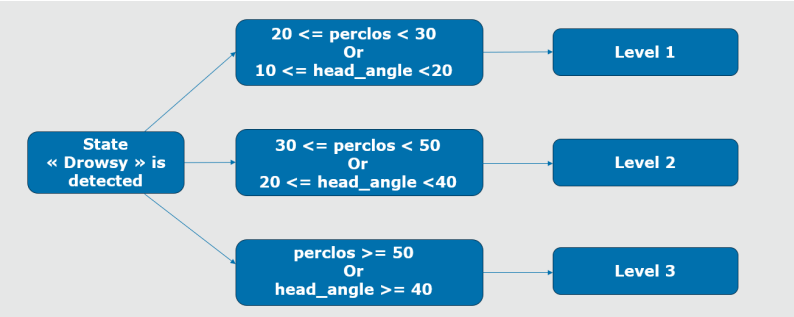


Figure 2.3: The detection levels of our program.

The 2.3 figure above give us an idea about how the drowsiness level variate accordinatly to the variations of perclos percentage and the head angle deviation.

2.3 Sub-Models Coding Logic

2.3.1 Perclos Calculation Coding

The PERCLOS calculation module is central to monitoring the driver's eye closure in real-time, which is a key indicator of drowsiness. Below are the key aspects of its implementation:

Detection and Processing

The module begins by utilizing the `dlib` library to detect facial landmarks within each frame captured from the video feed. The facial landmarks are crucial for identifying the positions of the eyes, which are then used to calculate the Eye Aspect Ratio (EAR). The detection process is initiated with the following lines:

```
detector = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")
```

These lines set up the face detector and the shape predictor, which are used to locate the eye landmarks on the detected face.

Buffered Calculation

The module then calculates the EAR for both eyes, which is a measure based on the distances between specific points around the eye. The formula for EAR is:

```
ear = (dist.euclidean(eye[1], eye[5]) + dist.euclidean(eye[2], eye[4]))/
      (2.0 * dist.euclidean(eye[0], eye[3]))
```

This equation takes into account the vertical and horizontal distances between the landmarks, and is used to determine whether the eyes are open or closed.

To keep track of the EAR over time, the module uses buffers, which are implemented as deques:

```
self.ear_buffer = deque(maxlen=self.frames_for_period)
self.eye_closed_buffer = deque(maxlen=self.frames_for_period)
```

These buffers store the EAR values and track whether the eyes are closed across a specified time period, allowing the system to calculate the PERCLOS metric. The percentage of time the eyes are closed is computed as:

```
closed_eyes_frames = sum(self.eye_closed_buffer)
perc = (closed_eyes_frames / len(self.eye_closed_buffer)) * 100
```

This code calculates the PERCLOS percentage, providing a clear metric of the driver's drowsiness level.

Real-Time Updates

The module processes each video frame in real-time, continuously updating the EAR buffer and computing the current PERCLOS value. This allows the system to detect changes in the driver's state immediately and provide timely alerts.

Implementation Insights

The combination of `dlib` for landmark detection and `OpenCV` for image processing provides an efficient method for real-time monitoring. The use of deques ensures that the system only considers the most recent data, making it both responsive and accurate in detecting drowsiness.

2.3.2 Posture Detection Coding

The Posture Detection module is designed to assess the driver's posture by calculating the head angle relative to the shoulders, using the MediaPipe library. Here's how it's implemented:

Pose Detection

The module leverages MediaPipe's pose estimation capabilities to detect and track human pose landmarks. It processes each video frame to identify key landmarks such as the nose and shoulders:

```
mp_pose = mp.solutions.pose
pose = mp_pose.Pose()
```

These lines initialize the MediaPipe Pose model, which is responsible for detecting the pose landmarks in the video frames.

Angle Calculation

Once the pose landmarks are detected, the module calculates the angle of the head relative to the shoulders. The angle is calculated using the following logic:

```
shoulder_midpoint = [(left_shoulder[0] + right_shoulder[0]) / 2,
                     (left_shoulder[1] + right_shoulder[1]) / 2]
ab = np.array(nose) - np.array(shoulder_midpoint)
bc = np.array([nose[0], nose[1] + 0.1]) - np.array(shoulder_midpoint)
cosine_angle = np.dot(ab, bc) / (np.linalg.norm(ab) * np.linalg.norm(bc))
angle = np.degrees(np.arccos(np.clip(cosine_angle, -1.0, 1.0)))
```

This code calculates the angle by first determining the midpoint between the shoulders, then calculating the vector from this midpoint to the nose, and finally computing the angle using the dot product and trigonometry. The angle provides a quantitative measure of the driver's head tilt, which can indicate drowsiness or distraction.

Continuous Monitoring and Alerts

Similar to the PERCLOS module, the Posture Detection module continuously monitors the driver's posture by updating the head angle for each frame. If the head angle deviates beyond a certain threshold, it may indicate drowsiness, prompting the system to trigger alerts.

Implementation Insights

The integration of MediaPipe with `OpenCV` allows for real-time posture detection and analysis. The system's ability to calculate head angles accurately and quickly makes it a reliable tool for detecting early signs of driver fatigue.

2.4 Integration of the Two Sub-Models

The integration of the PERCLOS and Posture Detection sub-models is essential for a comprehensive driver monitoring system that can assess both eye closure and posture deviation in real-time. This section outlines how these two components are combined into a unified application.

2.4.1 Application Overview

The integrated system is implemented as a PyQt5 application, which serves as the graphical user interface (GUI) for monitoring driver drowsiness and posture. The application captures video from a webcam, processes the frames to compute PERCLOS and head angle, and displays these metrics along with the drowsiness level on the screen.

2.4.2 User Interface and Video Processing

The user interface of the application includes a video display and labels that show the PERCLOS percentage, head angle, and the current drowsiness level. The interface is straightforward, designed to present the key metrics in a clear and accessible way.

Video frames are captured using a webcam, and each frame is processed to extract the relevant data. For posture detection, the system calculates the head angle based on the positions of key landmarks like the shoulders and nose:

```
rgb_frame, head_angle = process_frame_for_posture(frame)
self.head_angle_label.setText(f"Head Angle: {head_angle:.1f}°")
```

Simultaneously, the system calculates the PERCLOS value, which indicates the percentage of time the driver's eyes are closed:

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
ear, perc = self.perclos_calculator.process_frame_for_perclos(gray)
self.perclos_label.setText(f"PERCLOS: {perc:.2f}%")
```

2.4.3 Drowsiness Level Assessment

The application continuously evaluates the driver's state by comparing the PERCLOS percentage and head angle against predefined thresholds. Depending on these values, the system assigns a drowsiness level, which is then displayed on the interface:

```
if perc >= 50 or head_angle >= 40:
    level = "Level 3"
elif 30 <= perc < 50 or 20 <= head_angle < 40:
    level = "Level 2"
elif 20 <= perc < 30 or 10 <= head_angle < 20:
    level = "Level 1"
else:
    level = "None"
```

This logic ensures that the system can effectively detect and categorize the severity of the driver's drowsiness, providing appropriate alerts.

2.4.4 Real-Time Operation and Control

The application operates in real-time, with video frames being processed approximately 30 times per second. Users can start, pause, and stop the detection process using buttons on the interface. The system is designed to be user-friendly, ensuring that even users without technical expertise can operate it effectively.

2.4.5 Summary of Integration

The integration of the PERCLOS and Posture Detection sub-models within a single application allows for comprehensive monitoring of the driver's state. By combining real-time video processing with clear visual feedback, the system provides an effective tool for enhancing driver safety.

2.5 Global Integration with Yolo Pretrained Model

In this section, we discuss the functionality of integrating a pretrained YOLO model into the drowsiness detection system. This integration is crucial for real-time analysis and detection of drowsiness in drivers based on video inputs.

2.5.1 Functionality Overview

The system processes a video file to detect drowsiness levels by combining object detection with the YOLO model and the evaluation of drowsiness indicators, such as PERCLOS (percentage of eye closure) and head posture.

2.5.2 Drowsiness Detection Logic

The video input is analyzed frame by frame. The YOLO model is utilized to detect objects within each frame, while specialized algorithms assess the driver's drowsiness by calculating PERCLOS and evaluating head posture. The system categorizes the drowsiness level into three distinct stages based on these indicators.

2.5.3 Output and User Interaction

The processed video, containing visual indicators of drowsiness levels and detected objects, is saved and can be downloaded for further analysis. The system also allows for real-time feedback, providing critical information directly to the user to enhance safety measures during driving.

Chapter 3

Test and Simulation of Code Logic

3.1 Result of the Execution of the Two Sub-Models on a Local Machine

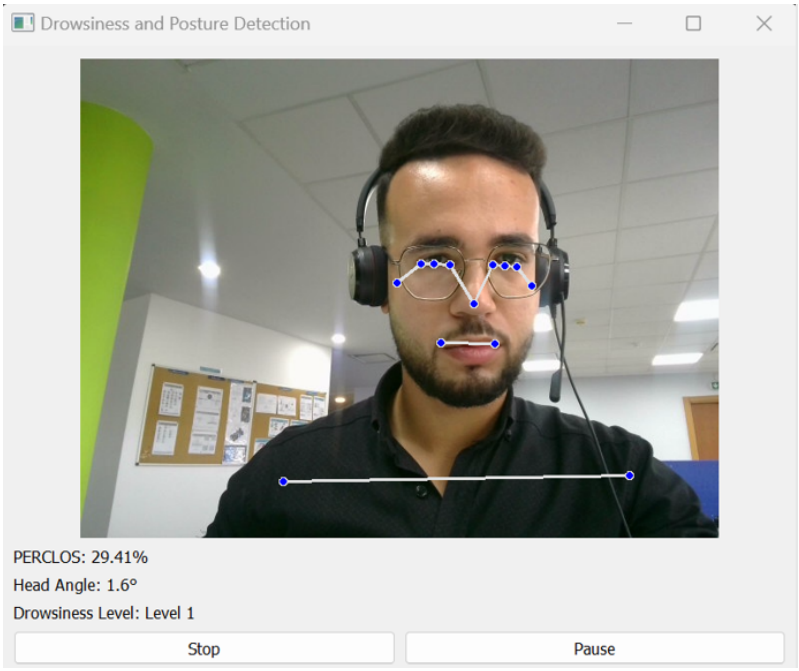


Figure 3.1: Testing result of the two submodule in graphical interface.

The figure above describe our first system test in our local machine, where we have run the two sub-models of "perclos_calculation" and "angle_deviation", to describe how this levels change accordinatly to the changing state of this values.

3.1.1 Drowsiness Level Indicator

The dynamic label located beneath the PERCLOS and Head Angle indicators displays the "Drowsiness Level" in real-time. The drowsiness level adjusts based on the values derived from the PERCLOS and head angle calculations. The levels are categorized as follows:

- **None**: No significant signs of drowsiness.
- **Level 1**: Mild drowsiness or slight posture deviation.

- **Level 2**: Moderate drowsiness or noticeable posture deviation.
- **Level 3**: Severe drowsiness or significant posture deviation.

3.1.2 Control Buttons

The interface includes several control buttons that allow the user to manage the detection process:

- **Start**: Initiates video capture and begins the drowsiness and posture detection process.
- **Pause/Resume**: Temporarily halts the video feed processing, with the option to resume.
- **Stop**: Ends the video capture, ceases all processing, and closes the application.

3.1.3 How It Works

The system operates as follows:

- When the user clicks the **Start** button, the webcam feed is activated, and the system begins processing each frame to detect eye closure and head posture.
- The **PERCLOS Calculator** monitors the Eye Aspect Ratio (EAR) and calculates the percentage of time the eyes are closed over a specified period. This value is continuously updated and displayed in the PERCLOS label.
- Concurrently, the **Posture Deviation Module** calculates the head angle based on detected landmarks and displays this value in the Head Angle label.
- The **Drowsiness Level** is dynamically determined based on predefined thresholds for both PERCLOS and head angle, with real-time updates shown on the interface.
- The user can pause or stop the detection process at any time by using the corresponding control buttons.

3.2 Result of Testing Global Program on Google Colab

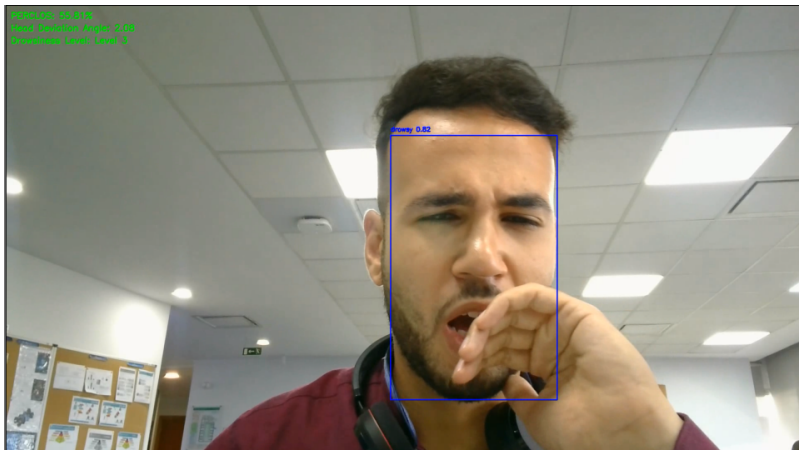


Figure 3.2: Testing result of the the global Program on Google colab.

This is an outputed video of an original test video that we have entered to our main Program on google colab.
We have used this strategie because we cannot open webcam with google colab and we cannot generate an interface.

3.2.1 Key Features

Object Detection

- The YOLO model (`best.pt`) identifies and labels objects within each frame of the video.
- Bounding boxes are drawn around detected objects, and each box is labeled with the object type and confidence score.

Drowsiness Detection

- **PERCLOS Analysis**: The percentage of eye closure is computed using the PERCLOS Calculator. This metric helps in assessing the level of drowsiness based on eye closure.
- **Posture Analysis**: Head posture is analyzed to calculate the deviation angle from the shoulders using the posture detection module. This helps in evaluating the head position, which can indicate drowsiness.

Drowsiness Level Indicator

The video displays real-time drowsiness levels based on both PERCLOS and head posture:

- **Level 1**: Mild drowsiness or slight posture deviation.
- **Level 2**: Moderate drowsiness or noticeable posture deviation.
- **Level 3**: Severe drowsiness or significant posture deviation.

- **None**: No significant signs of drowsiness.

3.2.2 Functionality

The video is analyzed frame-by-frame, with the YOLO model detecting objects, while PERCLOS and posture calculations are performed concurrently. Annotations are dynamically updated based on the analysis, providing real-time feedback on the drowsiness level and object presence in the video.

Chapter 4

Widescreen Development

4.1 Widescreen Functionalities

The widescreen module in our system is a graphical user interface (GUI) developed to serve as the Human-Machine Interface (HMI) for an automotive dashboard. This interface was built using PyQt5, a comprehensive set of Python bindings for the Qt application framework, which is widely used for developing cross-platform applications with graphical user interfaces.

The widescreen interface provides a user-friendly dashboard that displays various vehicle-related metrics and alerts, ensuring a seamless interaction between the driver and the vehicle. Below are the key functionalities of the widescreen:

4.1.1 User Interface Layout

The layout of the interface is designed to be both intuitive and informative:

- **Top Bar:** The top bar displays critical information such as the weather, temperature, current time, and system status icons. This ensures that the driver has quick access to essential data without distraction.
- **Map Frame:** The central area of the dashboard displays a real-time video capture of the driver's state. This area is also labeled with the "POSTURE PRO Team" and "MBSE Department" to emphasize the project's origins and contributors.
- **Right Rectangles:** These are dedicated to showing crucial metrics, including the PERCLOS percentage (an indicator of drowsiness based on eye closure), head angle deviation, and the overall drowsiness level. These metrics are displayed prominently to alert the driver when necessary.
- **Footer Buttons:** The bottom section includes navigation and control buttons that allow the driver to access various vehicle functions easily.

4.1.2 Drowsiness Alerts

The widescreen module also features an advanced drowsiness alert system. Depending on the severity of the detected drowsiness, the system displays different levels of alerts:

- **Level 1 Alert:** A dialog prompts the driver to rest if they are feeling tired. This is a gentle reminder to encourage the driver to take a break.

- **Level 2 Alert:** If the drowsiness level increases, a more urgent dialog appears, warning the driver to find the nearest rest area. This alert is designed to ensure the driver takes immediate action to avoid any potential risks.
- **Level 3 Alert:** At the highest level of drowsiness, the system issues a critical alert, urgently prompting the driver to stop. Additionally, the system integrates with the CARLA simulator to simulate a safe emergency stop of the vehicle on the side of the road, accompanied by the activation of hazard lights.

4.2 Used Tools

The development of the widescreen module involved several key tools and technologies:

- **PyQt5:** The primary framework used to develop the GUI. PyQt5 offers a robust set of tools to create desktop applications with a modern and professional appearance. It is highly suitable for creating complex interfaces like the automotive HMI dashboard.
- **QtCore and QtGui:** These PyQt5 modules were used extensively for handling core functionalities, such as timers and image processing. They enabled the seamless integration of various components within the interface, ensuring smooth operation and interaction.

4.3 Integration of Pop-Ups

The widescreen interface includes a set of pop-up dialogs designed to alert the driver to various levels of drowsiness. These pop-ups are integrated into the GUI as follows:

4.3.1 Dialog Management

Pop-up dialogs are dynamically managed based on the driver's drowsiness level. The system utilizes timers and event handlers to control the visibility and behavior of these dialogs. Each dialog corresponds to a specific level of drowsiness and prompts the driver with appropriate actions.

4.3.2 Alert Levels

As described earlier, there are three levels of drowsiness alerts, each triggering a different pop-up:

- **Level 1:** A dialog with two options—rest now or continue driving—is displayed when the system detects mild drowsiness. The driver's choice is recorded, and the system adjusts its behavior accordingly.
- **Level 2:** When moderate drowsiness is detected, a pop-up with a blinking effect warns the driver to stop at the nearest rest area. The pop-up's visibility toggles periodically to draw the driver's attention without being overly intrusive.

- **Level 3:** For severe drowsiness, the system shows a critical alert dialog, warning that the vehicle will take control if the driver does not respond. This dialog is accompanied by a timer that counts down to the vehicle's automatic intervention. The integration with CARLA ensures that this transition to an automated emergency stop is simulated accurately.

4.3.3 Blinking and Timers

To enhance the effectiveness of these alerts, blinking effects are used for the most critical alerts (Level 3). Timers control the blinking, ensuring the message is clear and urgent without causing excessive distraction.

4.3.4 User Interaction and Safety Measures

User interactions with these dialogs are designed to be simple and straightforward. For instance, clicking "I'll stop right now" in a Level 1 alert will immediately close the dialog and stop further alerts. In contrast, choosing "I'm not tired" resets the drowsiness metrics but keeps the system on high alert for further signs of fatigue.

The system also ensures that all dialogs are properly closed when the main window is closed, maintaining a clean and error-free operation throughout.

4.4 Test and Simulation

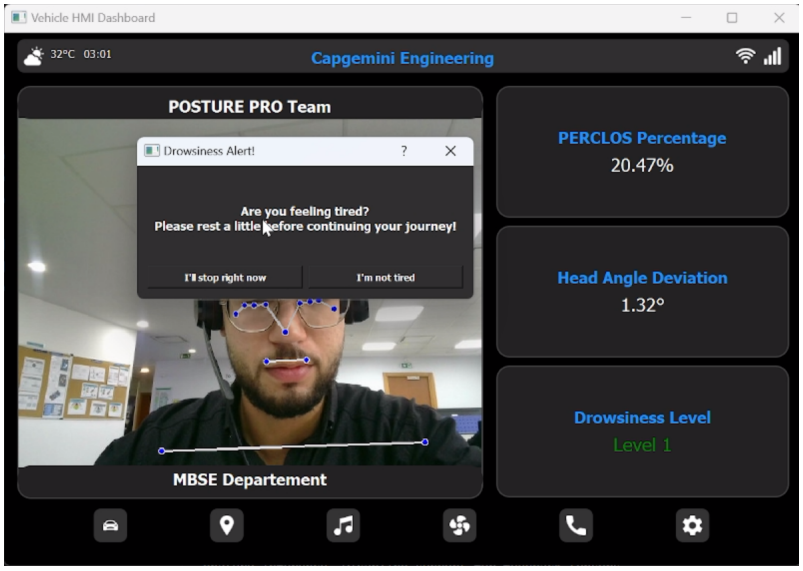


Figure 4.1: Widescreen to ensure communication with the driver and alert him.

The interface is designed to assist drivers by providing real-time feedback based on their drowsiness levels. It integrates a set of alerts and dialogs to ensure the driver's safety. The interface includes dynamic visual elements that respond to the driver's drowsiness metrics, such as PERCLOS and head deviation.

4.4.1 Alerts and Dialogs

Level 1 Alert

Dialog Box: When the driver is detected to be at Level 1 drowsiness, a dialog box appears prompting the driver to rest if they feel tired.

Buttons: The dialog includes two options:

- **"I'll stop right now"**: If selected, the system will register the driver's decision to stop immediately.
- **"I'm not tired"**: If chosen, the system resets the PERCLOS and drowsiness metrics. The dialog will not reappear unless the drowsiness level transitions back to Level 1.

Level 2 Warning

Dialog Box: At Level 2, a warning dialog is displayed, urging the driver to find a rest area immediately.

Blinking Effect: The warning dialog features a blinking effect to attract the driver's attention and emphasize the urgency of the situation.

Level 3 Emergency

Dialog Box: For Level 3 drowsiness, an emergency dialog box appears with an urgent message that the vehicle will take control in 10 seconds.

Blending Text: The text in the emergency dialog box blinks in red, highlighting the critical nature of the warning and prompting immediate action from the driver.

Chapter 5

Integration on CARLA Simulator

5.1 Tools Explanation



Figure 5.1: Carla simulator logo.

The CARLA (Car Learning to Act) simulator is an open-source autonomous driving simulator designed for the development and testing of autonomous vehicle systems. It provides a high-fidelity, realistic virtual environment with a variety of urban and rural settings. CARLA supports various driving scenarios, weather conditions, and traffic systems, allowing for comprehensive testing of driving algorithms and vehicle behavior. Its modular architecture facilitates the integration of custom sensors, control systems, and scenario setups, making it a valuable tool for researchers and developers in the field of autonomous driving.

it's going to be a relevant tool in our case to simulate the emergency stop of our vehicle on the level 3.
In our simulation where going to use "Town 03" as our simulation environment

5.2 Integration with our main system



Figure 5.2: test vehicle on autopilot mode in Carla environment.

Our carla program that's you car found it on the implementation folder as "test_carla.py" is used to control a simulated car in a virtual environment provided by CARLA, a driving simulation software. Here's a straightforward explanation of what the code does:

5.2.1 Connecting to the Simulation

Our program starts by connecting to the CARLA simulation running on your computer. It sets up the necessary settings to communicate with the simulation environment.

5.2.2 Setting Up the Simulation

It then prepares the simulation by removing any existing cars from the scene. This ensures that the simulation starts fresh without any previous vehicles interfering.

5.2.3 Spawning a New Car

The code selects a car model from the simulation's library and places it near a specific location in the virtual world, such as a roundabout.

5.2.4 Automatic Driving

The car is set to drive automatically (using autopilot). The code also adjusts some settings to make sure the car follows traffic rules and drives safely.

5.2.5 User Interaction

The program waits for you to type a command. If you type 'v', the car will stop and turn on its hazard lights (the blinking lights used to signal danger or caution). This command is going to be replaced by a value giving by our main program to change the value to "True", when the drowsiness level reach "level 3".

5.2.6 Stopping the Car

When you type 'v', the car will:

- Stop immediately.
- Turn off the autopilot.
- Activate hazard lights, which blink to indicate the car is stopped in a potentially dangerous situation.

5.2.7 Ending the Simulation

If you stop the simulation (using a keyboard shortcut like Ctrl + C), the code makes sure the car is properly removed from the simulation environment to avoid leaving any debris.

5.2.8 Specifications

- **Vehicle Model:** Uses a random car model from the CARLA simulation library.
- **Location:** The car is placed near a roundabout.
- **Autopilot:** Enabled to drive the car automatically.
- **Traffic Management:** Configures traffic rules to ensure safe driving.
- **User Command:** Allows stopping the car and activating hazard lights via a simple text input.
- **Cleanup:** Ensures the car is properly removed from the simulation when done.

5.3 multi-threading issue

During the simulation, an error may occur when stopping the vehicle and activating hazard lights, specifically the 'PyEval_SaveThread Carla' error. This error can arise due to several reasons related to multi-threading and integration between Python and the CARLA simulator.

```
D:\WindowsNoEditor\PythonAPI\examples>python test_code.py
Vehicle spawned with custom name: Capgemini Engineering
Voulez-vous arrêter la voiture? (Entrez 'v' pour arrêter): v
Vehicle stopped and hazard lights activated.
Fatal Python error: PyEval_SaveThread: NULL tstate
```

Figure 5.3: multi-threading error.

5.3.1 Error Overview

The 'PyEval_SaveThread' error is related to the interaction between Python's Global Interpreter Lock (GIL) and the threading model used by CARLA. This issue typically occurs when there are conflicts in thread management between the Python interpreter and the CARLA API.

5.3.2 Potential Problems

- **CPU Power:** Insufficient CPU resources or high system load can lead to performance issues and threading conflicts. The CARLA simulator and the Python script may compete for CPU time, leading to errors like 'PyEval_SaveThread'. Ensuring that your system has adequate processing power and minimizing background tasks can help alleviate this issue.

- **CARLA Version:** Compatibility issues between different versions of CARLA and the Python API can also cause threading problems. Ensure that you are using a version of CARLA that is compatible with your Python setup. Updating to the latest version of CARLA or the Python API might resolve these issues if they are known to be related to specific versions.

- **Modeling the Vehicle with MATLAB/Simulink:** Integrating MATLAB/Simulink models with CARLA might introduce additional complexity in thread management. If you are using MATLAB/Simulink to model the vehicle or simulate its behavior, ensure that the integration is correctly set up and that it adheres to CARLA's requirements for external interfacing. Improper integration or synchronization between MATLAB/Simulink and CARLA can lead to threading issues.

5.3.3 Recommendations

To address the 'PyEval_SaveThread' error and minimize the risk of encountering similar issues, consider the following recommendations:

- Monitor and manage CPU usage to ensure that sufficient resources are available for both the CARLA simulator and the Python script.

- Verify that you are using compatible versions of CARLA and the Python API. Check the documentation for any known issues or updates that may address threading problems.

- Ensure proper integration between MATLAB/Simulink models and CARLA, following best practices for interfacing and synchronization. Test the integration thoroughly to avoid conflicts.

Conclusion

The integration of advanced drowsiness detection algorithms with a high-fidelity simulation environment has proven effective in developing a robust driver assistance system capable of real-time intervention. By combining PERCLOS, posture detection, and object recognition, the system provides a comprehensive approach to monitoring driver fatigue and initiating safety measures, such as emergency stops. The use of the CARLA simulator was instrumental in testing the system under various conditions, highlighting both its strengths and areas for improvement. Challenges such as multi-threading issues and compatibility concerns were addressed, underscoring the importance of careful resource management and adherence to best practices in system integration. Overall, the findings affirm the value of simulation tools in advancing autonomous driving technologies and demonstrate the potential of this integrated system to significantly enhance road safety. Future work will focus on refining the system's performance and exploring additional scenarios to further improve its reliability and responsiveness.